

Nombre	Manuel Fernando Guzmán Arias	ID	4948560
Carrera	Ingeniería de Software	Fecha	2026-10-01

Repositorio: <https://github.com/FerGuzman65/Lab5-Pruebas-de-software>

LABORATORIO # 5 – PRUEBAS DE CAJA GRIS

Introducción

En este laboratorio, los estudiantes aplicarán técnicas de pruebas de caja gris para analizar y validar un programa de gestión de calificaciones, utilizando información parcial sobre su estructura interna, componentes y dependencias.

A través de esta práctica, los estudiantes desarrollarán habilidades en:

1. Analizar parcialmente la estructura interna del programa, utilizando el conocimiento de sus componentes, atributos, funciones y dependencias para diseñar casos de prueba mediante pytest.
2. Diseñar pruebas a partir del conocimiento de la configuración y funcionamiento interno del sistema, modificando temporalmente atributos, elementos de configuración y dependencias mediante herramientas como monkeypatch.
3. Aplicar técnicas de aislamiento de componentes, mediante mecanismos como `setattr()`, `setitem()` y `delattr()`, con el propósito de controlar temporalmente el comportamiento de determinados elementos durante las pruebas.

Descripción del programa

El programa implementa un sistema simplificado para la gestión y procesamiento de órdenes de compra. El sistema permite calcular el subtotal de una orden, determinar descuentos de acuerdo con el monto de la compra, calcular impuestos y procesar el pago mediante una representación de un servicio externo.

El programa está compuesto por las clases `Order`, `DiscountService`, `PaymentGateway` y `OrderService`, además de una configuración global que contiene la tasa de impuesto y la moneda utilizada.

Requerimientos Funcionales y No Funcionales

ID	Requerimiento funcional
RF-01	El sistema deberá permitir crear una orden de compra a partir de una lista de productos.

ID	Requerimiento funcional
RF-02	El sistema deberá almacenar los productos asociados a una orden, incluyendo nombre, precio y cantidad.
RF-03	El sistema deberá calcular el subtotal de una orden multiplicando el precio de cada producto por su cantidad.
RF-04	El sistema deberá calcular un descuento del 10 % cuando el subtotal sea mayor o igual a \$1000.
RF-05	El sistema deberá calcular un descuento del 5 % cuando el subtotal sea mayor o igual a \$500 y menor que \$1000.
RF-06	El sistema deberá aplicar un descuento de 0 % cuando el subtotal sea menor que \$500.
RF-07	El sistema deberá obtener la tasa de impuesto definida en la configuración del sistema.
RF-08	El sistema deberá calcular el impuesto sobre el subtotal después de aplicar el descuento.
RF-09	El sistema deberá calcular el total de la orden sumando el impuesto al subtotal después del descuento.
RF-10	El sistema deberá procesar el pago utilizando el monto total calculado para la orden.
RF-11	El sistema deberá permitir la sustitución del servicio de pagos durante las pruebas del sistema.
RF-12	El sistema deberá continuar el cálculo de la orden sin aplicar descuento cuando el servicio de descuentos no esté disponible.
RF-13	El sistema deberá retornar los resultados del cálculo de la orden, incluyendo subtotal, descuento, impuesto y total.
RF-14	El sistema deberá retornar el resultado del procesamiento del pago junto con la información calculada de la orden.

Actividades

Instrucciones generales

Las siguientes actividades se llevarán a cabo **de manera sincronizada con el profesor**, quien explicará el procedimiento y las ventajas de cada paso:

1. Abre el proyecto, ya sea descargando el archivo.py o haciendo un fork al repositorio proporcionado, y revisa las funciones disponibles en los archivos proporcionados
2. Diseña los casos de prueba de acuerdo a las actividades.

3. Ejecuta las pruebas sobre el software asignado.
4. Documenta los resultados en las tablas que se muestran en la siguiente sección .
5. Realiza un commit por casos de pruebas es decir no es forzoso por cada caso para registrar tus resultados y observaciones en GitHub.

Actividad

- **Diseñar un conjunto de casos de prueba para utilizar las técnicas de monkeypatch**

ID	CP-01
Método de monkeypatch	monkeypatch.setattr()
Entradas	Orden con dos productos: Teclado \$800.00 x 1 y Mouse \$300.00 x 1. CONFIG["tax_rate"] = 0.16 (valor original de la configuración). Se sustituye el atributo payment_gateway de la instancia de OrderService por un doble de prueba (FakePaymentGateway), porque la API real del servicio de pagos todavía no está disponible.
Resultado esperado	Subtotal: 1100 Descuento (10% por ser mayor o igual a 1000): 110 Impuesto (16% sobre 990): 158.4 Total: 1148.4 El resultado del pago regresa status "approved" y el gateway recibe un solo cobro por 1148.4.
Resultado obtenido	Igual al resultado esperado. Subtotal 1100, descuento 110, impuesto 158.4, total 1148.4. El pago regresa status "approved", con una sola llamada al gateway por 1148.4. test_shop.py::test_order_normal PASSED

ID	CP-02
-----------	--------------

Método de monkeypatch	monkeypatch.setitem()
Entradas	La misma orden: Teclado \$800.00 x 1 y Mouse \$300.00 x 1. Se modifica la llave "tax_rate" del diccionario CONFIG, de 0.16 a 0.08, sólo durante esta prueba. Se sustituye también el payment_gateway por el doble de prueba, para poder procesar la orden completa y revisar el monto que se cobra. La técnica que se prueba en este caso es setitem; el doble sólo sirve de apoyo.
Resultado esperado	Subtotal: 1100 Descuento (10%): 110 Impuesto (8% sobre 990): 79.2 Total: 1069.2 El gateway recibe un cobro por 1069.2, es decir, la tasa modificada llega hasta el monto cobrado. Al terminar la prueba, CONFIG debe quedar otra vez con tax_rate en 0.16.
Resultado obtenido	Igual al resultado esperado. Subtotal 1100, descuento 110, impuesto 79.2, total 1069.2. El pago regresa status "approved" y el gateway recibe un cobro por 1069.2. La configuración se restauró sola: el caso CP-03 se ejecuta después y vuelve a calcular el impuesto con 0.16, lo que confirma que el cambio no se quedó guardado. test_shop.py::test_tax_rate_from_config PASSED

ID	CP-03
Método de monkeypatch	monkeypatch.delattr()
Entradas	La misma orden: Teclado \$800.00 x 1 y Mouse \$300.00 x 1. CONFIG["tax_rate"] = 0.16. Se elimina el atributo discount_service de la instancia de

	OrderService, para simular que el servicio de descuentos no está disponible.
Resultado esperado	Subtotal: 1100 Descuento: 0 (no hay servicio de descuentos) Impuesto (16% sobre 1100): 176 Total: 1276 El cálculo de la orden debe terminar sin error, según el RF-12.
Resultado obtenido	Igual al resultado esperado. Subtotal 1100, descuento 0, impuesto 176.0, total 1276.0. El cálculo continuó sin descuento y sin lanzar ningún error. test_shop.py::test_order_without_discount_service PASSED

Documentación

Entrega de este reporte de laboratorio

Agregar sección de anexos de capturas de pantalla por cada caso de prueba

Anexos

Se incluye la evidencia de cada caso de prueba: el código con el que se diseñaron y la salida de la consola al ejecutarlos con pytest. Los commits de cada caso están en el repositorio indicado en la primera página.

Anexo A. Código de los casos de prueba (test_shop.py)

```
from shop import Order
from shop import OrderService
from shop import CONFIG
import pytest
```

```
class FakePaymentGateway:
    """Doble de prueba que sustituye al servicio externo de pagos (RF-11).
```

```
    La API real no esta disponible, por lo que se aisla el componente y
    se registra el monto cobrado para poder verificarlo.
```

```
"""

def __init__(self):
    self.charged_amount = None
    self.calls = 0

def charge(self, amount):
    self.calls += 1
    self.charged_amount = amount

    return {
        "status": "approved",
        "amount": amount,
        "currency": CONFIG["currency"]
    }

@pytest.fixture
def order():
    products = [
        {
            "name": "Teclado",
            "price": 800,
            "quantity": 1
        },
        {
            "name": "Mouse",
            "price": 300,
            "quantity": 1
        }
    ]

    return Order(products)

# CP-01 | monkeypatch.setattr() | RF-10, RF-11, RF-14
# Sustituye la dependencia payment_gateway por un doble de prueba.
# Entradas: Teclado $800 x1, Mouse $300 x1 | tax_rate = 0.16
# Esperado: subtotal 1100, descuento 110, impuesto 158.4, total 1148.4
#           y pago aprobado por el monto total de la orden.
def test_order_normal(monkeypatch, order):

    service = OrderService(CONFIG)

    fake_gateway = FakePaymentGateway()

    monkeypatch.setattr(
        service,
        "payment_gateway",
        fake_gateway
    )

    order_processed = service.process_order(order)

    assert order_processed["payment"]["status"] == "approved"
    assert order_processed["order"]["subtotal"] == 1100
```

```
assert order_processed["order"]["discount"] == 110
assert order_processed["order"]["tax"] == 158.4
assert order_processed["order"]["total"] == 1148.4

assert fake_gateway.calls == 1
assert fake_gateway.charged_amount == 1148.4

# CP-02 | monkeypatch.setitem() | RF-07, RF-08, RF-09
# Modifica temporalmente la configuracion global para probar el sistema
# con otra tasa de impuesto, sin alterar CONFIG para los demas casos.
# El doble de pagos solo sirve de apoyo: la tecnica que se prueba aqui es
# setitem, y se usa process_order para comprobar que la tasa modificada
# llega hasta el monto que se cobra.
# Entradas: Teclado $800 x1, Mouse $300 x1 | tax_rate = 0.08
# Esperado: subtotal 1100, descuento 110, impuesto 79.2, total 1069.2
def test_tax_rate_from_config(monkeypatch, order):

    monkeypatch.setitem(CONFIG, "tax_rate", 0.08)

    service = OrderService(CONFIG)

    fake_gateway = FakePaymentGateway()

    monkeypatch.setattr(
        service,
        "payment_gateway",
        fake_gateway
    )

    order_processed = service.process_order(order)

    result = order_processed["order"]

    assert result["subtotal"] == 1100
    assert result["discount"] == 110
    assert result["tax"] == 79.2
    assert result["total"] == 1069.2

    assert order_processed["payment"]["status"] == "approved"
    assert fake_gateway.charged_amount == 1069.2

# CP-03 | monkeypatch.delattr() | RF-12
# Elimina la dependencia discount_service para simular que el servicio
# de descuentos no esta disponible; el calculo debe continuar sin descuento.
# Entradas: Teclado $800 x1, Mouse $300 x1 | tax_rate = 0.16
# Esperado: subtotal 1100, descuento 0, impuesto 176.0, total 1276.0
def test_order_without_discount_service(monkeypatch, order):

    service = OrderService(CONFIG)

    monkeypatch.delattr(
        service,
        "discount_service"
    )
```

```
result = service.calculate_total(order)

assert result["subtotal"] == 1100
assert result["discount"] == 0
assert result["tax"] == 176.0
assert result["total"] == 1276.0
```

Anexo B. Ejecución de los tres casos de prueba

```
===== test session starts =====
platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 --
C:\Users\manue\AppData\Local\Programs\Python\Python39\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\manue\OneDrive - Universidad Autonoma de Guadalajara\UAG\Pruebas de
software\Parcial 2\Lab5-Pruebas-de-software
collecting ... collected 3 items

test_shop.py::test_order_normal PASSED [ 33%]
test_shop.py::test_tax_rate_from_config PASSED [ 66%]
test_shop.py::test_order_without_discount_service PASSED [100%]

===== 3 passed in 0.01s =====
```

Anexo C. Ejecución individual de cada caso de prueba

```
===== test_order_normal =====
===== test session starts =====
collecting ... collected 1 item

test_shop.py::test_order_normal PASSED [100%]

===== 1 passed in 0.01s =====
===== test_tax_rate_from_config =====
===== test session starts =====
collecting ... collected 1 item

test_shop.py::test_tax_rate_from_config PASSED [100%]

===== 1 passed in 0.01s =====
===== test_order_without_discount_service =====
===== test session starts =====
collecting ... collected 1 item

test_shop.py::test_order_without_discount_service PASSED [100%]

===== 1 passed in 0.01s =====
```

Conclusiones

Escribe aquí tus conclusiones sobre lo aprendido en el laboratorio.

Las pruebas de caja gris permitieron probar el sistema sin modificar su código. Los tres casos se resolvieron desde el archivo de pruebas, aprovechando lo que ya se sabía de la estructura interna del programa: que `payment_gateway` y `discount_service` son atributos de la instancia de `OrderService`, y que la tasa de impuesto se lee de un diccionario de configuración.

Lo más útil fue entender que `monkeypatch` deshace los cambios al terminar cada prueba. Eso se ve en el CP-02: se cambia la tasa de impuesto a 0.08 y el caso siguiente vuelve a calcular con 0.16 sin tener que restaurar nada a mano. Si el valor se hubiera asignado directo al diccionario, el cambio se habría quedado y habría afectado a las demás pruebas.

También quedó claro por qué el programa lee `discount_service` con `getattr` y un valor por defecto en lugar de usarlo directo. Así se puede eliminar esa dependencia durante una prueba y el cálculo continúa sin descuento, que es justo el comportamiento que pide el RF-12.

El caso del pago mostró la diferencia entre corregir el programa y aislarlo. El método `charge` no devuelve nada, por lo que `process_order` entrega `None` en la parte del pago. En lugar de modificar `shop.py` se sustituyó el servicio por un doble de prueba, porque la API real todavía no existe y lo que se está probando es el cálculo de la orden, no el cobro.
